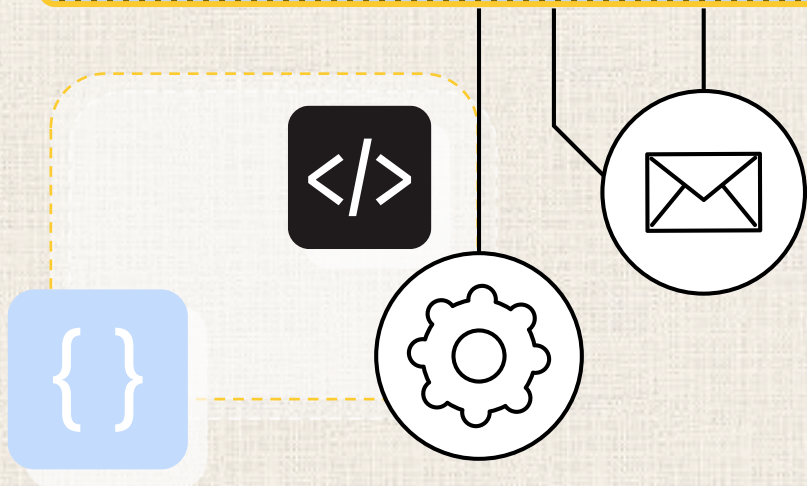


11강

다차원 구조

첨단공학부
정재화 교수





1. 2차원 리스트의 이해

- ✓ 리스트와 차원
- ✓ 2차원 리스트의 정적/동적 생성
- ✓ 2차원 리스트의 순회

2. 다차원 리스트의 이해

- ✓ 다차원 리스트의 개념
- ✓ 다차원 리스트의 순회



학습하기

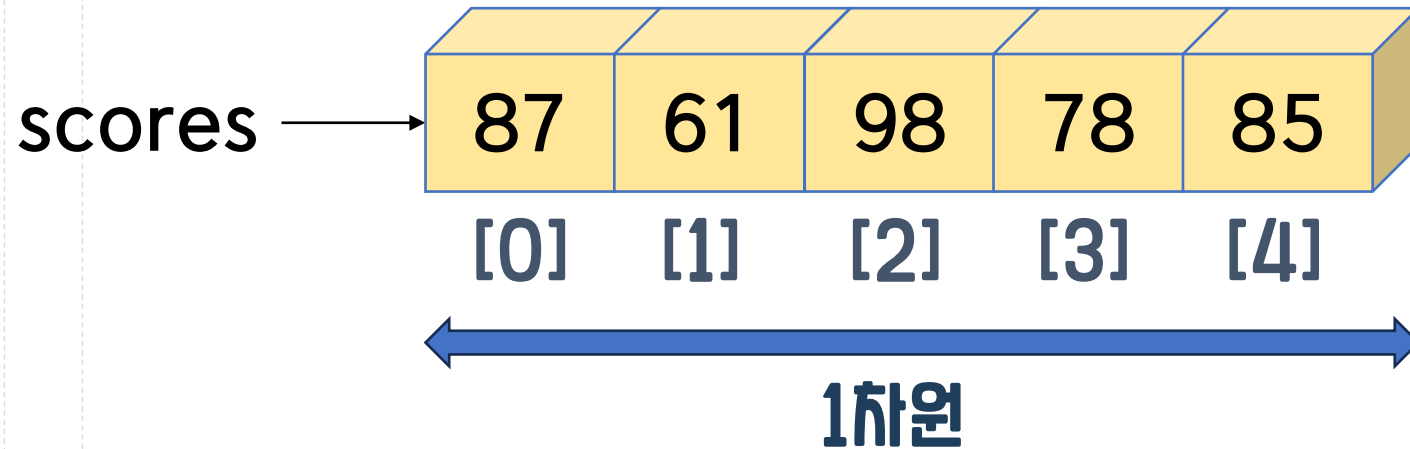
1

2차원 리스트의 이해

리스트와 차원

여러 개의 데이터(값)를 순서대로 묶어서 저장하는 자료구조

- ✓ 접근 연산자에 1개의 인덱스 값을 조절하여
데이터값에 접근



- ✓ 리스트에 다른 타입의 컬렉션을 원소로 삽입 가능
and vice versa

차원의 확장

2차원

1차원

거리표	서울	대전	광주	대구	울산	전주	순천	부산
서울	0	151	290	282	396	210	320	390
대전	151	0	178	148	259	90	235	260
광주	290	178	0	219	327	105	93	267
대구	282	148	219	0	87	198	218	130
울산	396	259	327	87	0	308	199	65
전주	210	90	105	198	308	0	141	265
순천	320	235	93	218	199	141	0	199
부산	390	260	267	130	65	265	199	0

다차원 리스트의 개념

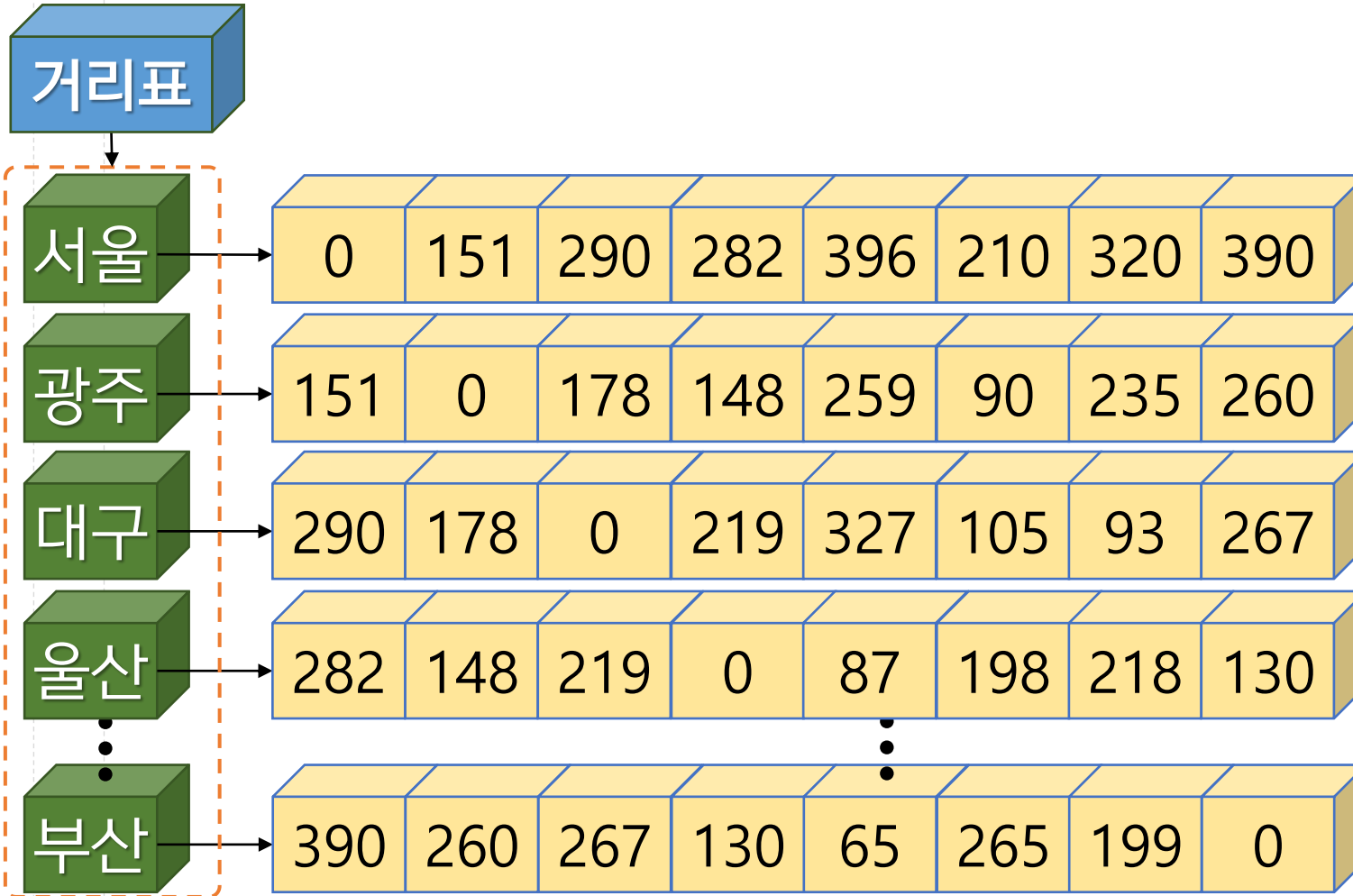
■ 리스트 내부에 다른 리스트가 원소로 내포된 리스트

- ✓ 행(row)과 열(column)로 이루어진
2차원 테이블 형태

■ 관련된 데이터를 하나의 변수로 묶어 논리적으로 그룹화

- ✓ 데이터 분석: 엑셀 스프레드시트, 행렬 데이터 처리
- ✓ 게임: 체스판, 지뢰찾기, 2D 게임 맵 좌표
- ✓ 이미지: 픽셀 데이터

2차원 리스트 구조



2차원 리스트 생성

구문형식

list([원소 시퀀스₁],
 [원소 시퀀스₂],
 ⋮
 [원소 시퀀스_n]) 또는 [[원소 시퀀스₁],
 [원소 시퀀스₂],
 ⋮
 [원소 시퀀스_n]]

```
matrix = [[1, 2, 3, 4, 5],
           [6, 7, 9, 0, 1],
           [9, 8, 7, 6, 5],
           [1, 3, 5, 7, 9],
           [2, 4, 6, 8, 0]]
```

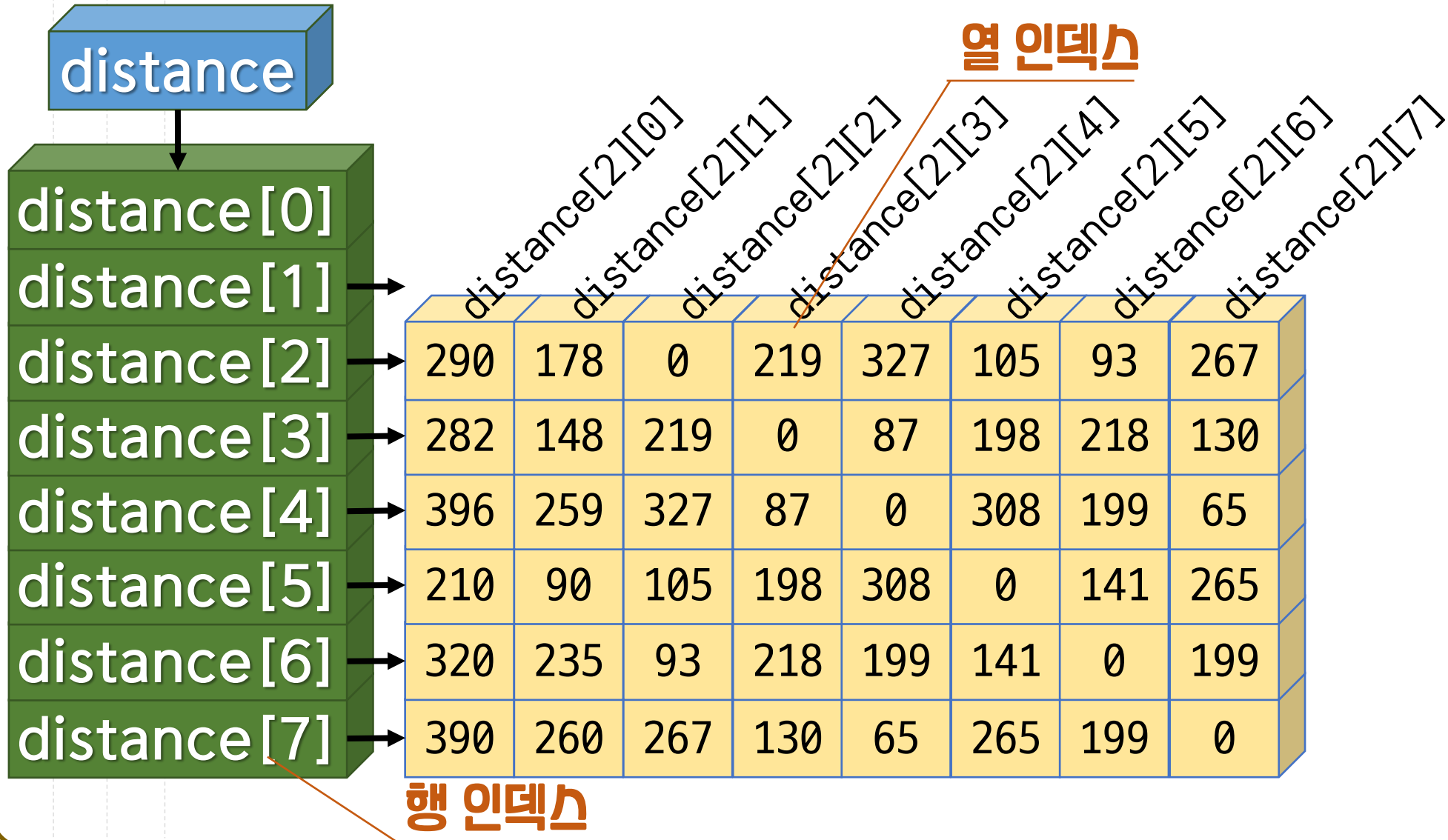
matrix

1	2	3	4	5
6	7	9	0	1
9	8	7	6	5
1	3	5	7	9
2	4	6	8	0

2차원 리스트 접근

	distance[0][0]	distance[0][1]	distance[0][2]	distance[0][3]	distance[0][4]	distance[0][5]	distance[0][6]	distance[0][7]
distance[0]	0	151	290	282	396	210	320	390
distance[1]	151	0	178	148	259	90	235	260
distance[2]	290	178	0	219	327	105	93	267
distance[3]	282	148	219	0	87	198	218	130
distance[4]	396	259	327	87	0	308	199	65
distance[5]	210	90	105	198	308	0	141	265
distance[6]	320	235	93	218	199	141	0	199
distance[7]	390	260	267	130	65	265	199	0

2차원 리스트 접근



2차원 리스트의 접근

구문형식

리스트이름[행인덱스][열인덱스]

```
print(matrix[1])
print(matrix[4])
print(matrix[0][0])
print(matrix[3][2])
```

```
[6, 7, 9, 0, 1]
[2, 4, 6, 8, 0]
1
5
```

matrix

1	2	3	4	5
6	7	9	0	1
9	8	7	6	5
1	3	5	7	9
2	4	6	8	0

2차원 리스트 동적 생성

특정 값(랜덤값, 사용자입력 등)으로 생성

```
import random as rd
```

```
# 행의 개수 nRows, 열의 개수 nColumns
```

```
distance = []
```

```
for i in range(nRows):
```

```
    row = []
```

```
        for j in range(nColumns):
```

```
            row.append(rd.randint(0, 99))
```

```
        distance.append(row)
```

```
print(distance)
```

2차원 리스트 순회

■ 리스트에 포함된 모든 데이터를 탐색

- ✓ 각 차원 순회하기 위한 2개의 반복 구조가 중첩

출발 도시 변수

```
for departure in distance:  
    for destination in departure:  
        print(destination, end = " ")  
    print()
```

도착 도시 변수

I 두 도시간 거리 출력하기

■ distance 리스트를 사용하여 사용자로부터 입력받은 도시 간 거리를 출력하는 프로그램

거리를 계산할 두 도시를 입력하세요:서울,대전
서울와/과 대전 사이의 거리는 151km입니다.

거리를 계산할 두 도시를 입력하세요:부산,광주
부산와/과 광주 사이의 거리는 267km입니다.

합계가 가장 큰 행 찾기

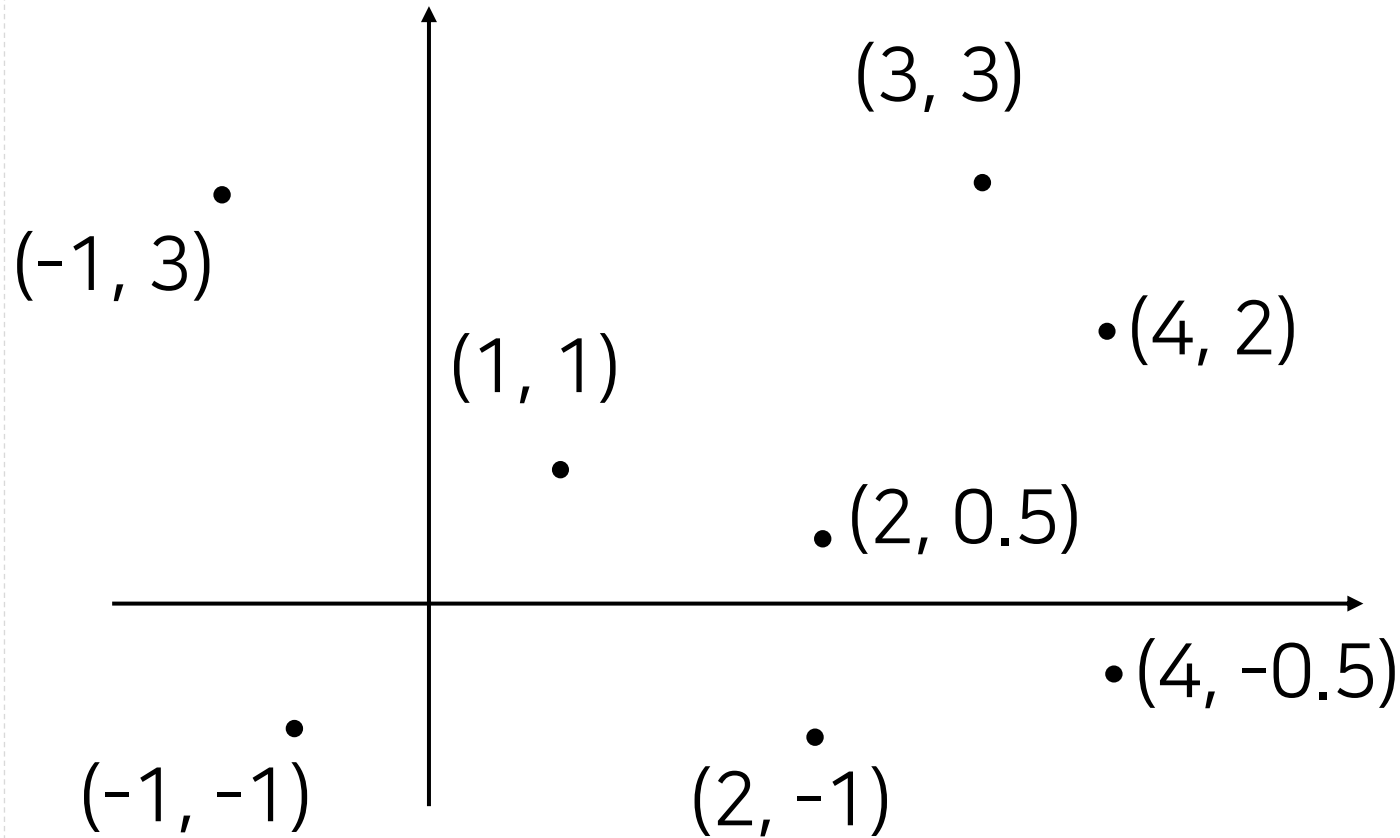
- 주어진 2차원 리스트에서 각 행에 포함된 원소의 합계가 가장 큰 행과 합계를 출력하시오.

matrix

1	2	3	4	5
6	7	9	0	1
9	8	7	6	5
1	3	5	7	9
2	4	6	8	0

가장 가까운 짝 찾기 문제

- 주어진 좌표축에서의 점 리스트에서 가장 가까운 두 점을 찾는 프로그램을 작성하시오.



학습하기

2

다차원 리스트의 이해

다차원 리스트의 개념

n-1 차원의 리스트가 복수 개 집합된 형태의 리스트

한국

distance[0]

distance[1]

distance[2]

distance[3]

distance[4]

distance[5]

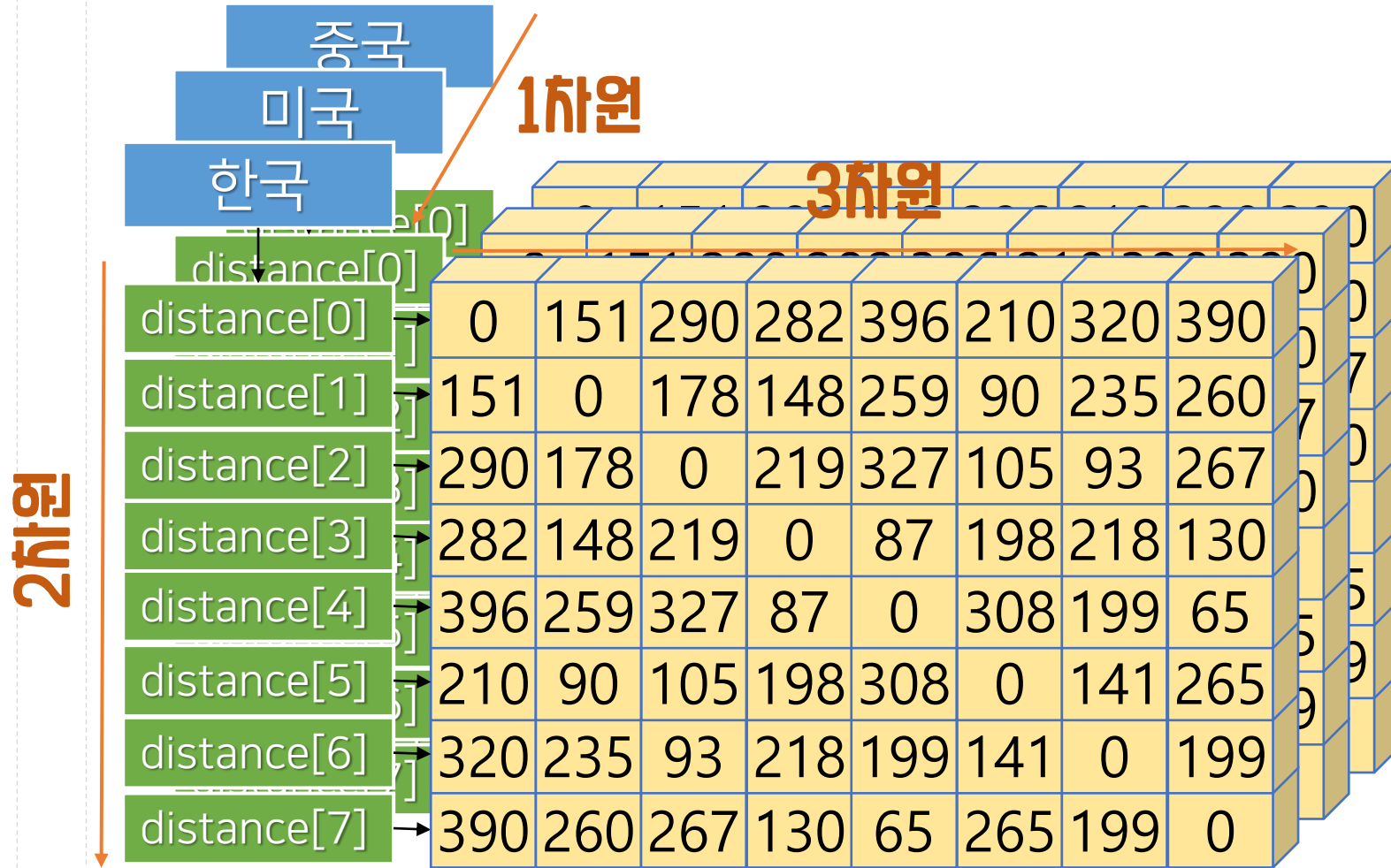
distance[6]

distance[7]

0	151	290	282	396	210	320	390
151	0	178	148	259	90	235	260
290	178	0	219	327	105	93	267
282	148	219	0	87	198	218	130
396	259	327	87	0	308	199	65
210	90	105	198	308	0	141	265
320	235	93	218	199	141	0	199
390	260	267	130	65	265	199	0

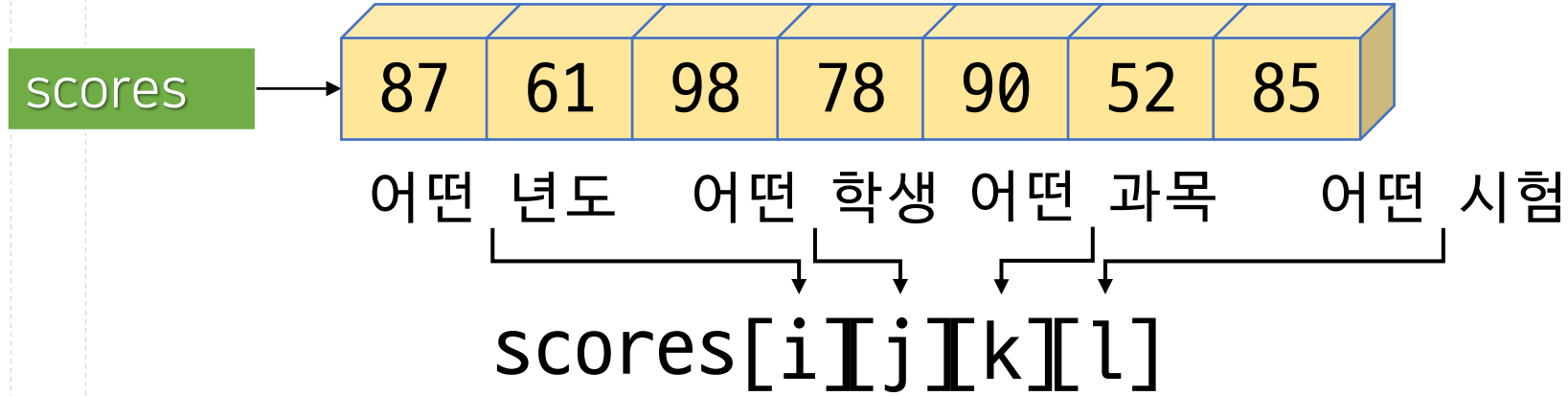
다차원 리스트의 개념

n-1 차원의 리스트가 복수 개 집합된 형태의 리스트

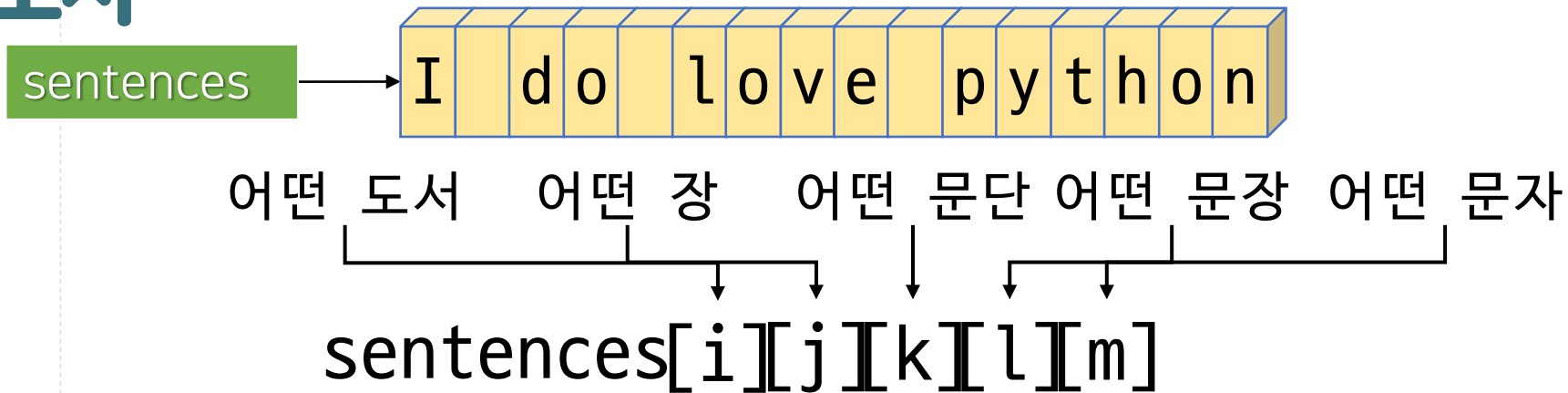


다차원 리스트의 예

시험 점수



도서



3차원 리스트 생성과 접근

3차원 리스트 생성

구문형식

```
[[ [원소 시퀀스11],  
  [원소 시퀀스21],  
  ⋮  
  [원소 시퀀스n1] ],  
 ...  
[[ [원소 시퀀스1m],  
  [원소 시퀀스2m],  
  ⋮  
  [원소 시퀀스nm] ]]
```

3차원 리스트 접근

구문형식

```
리스트이름[깊이인덱스][행인덱스][열인덱스]
```

3차원 리스트 순회

- 각 차원 순차적으로 순회하기 위한 n개의 반복 구조를 내포

국가 변수

출발 도시 변수

```
for country in distances:  
    for departure in country:  
        for destination in departure:  
            print(destination, end = " ")  
        print()  
    print()
```

도착 도시 변수

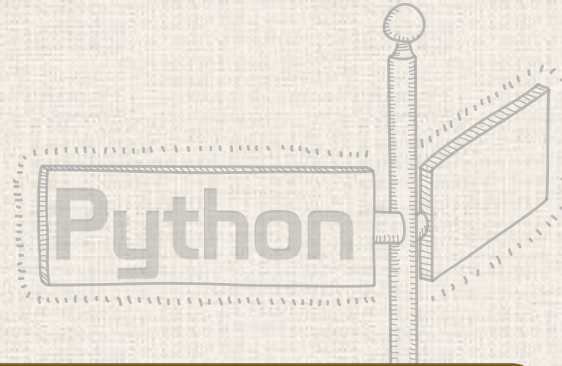
I 그레이스케일 이미지로 변경

- 주어진 방송통신대 픽셀 이미지를
흑백의 톤으로 변경하시오.
(힌트, RGB 값을 평균값으로 변환)

KNOW

KNOW

다음시간에는



11강. 다차원 구조

▶ 12강. 모듈

13강. 파일